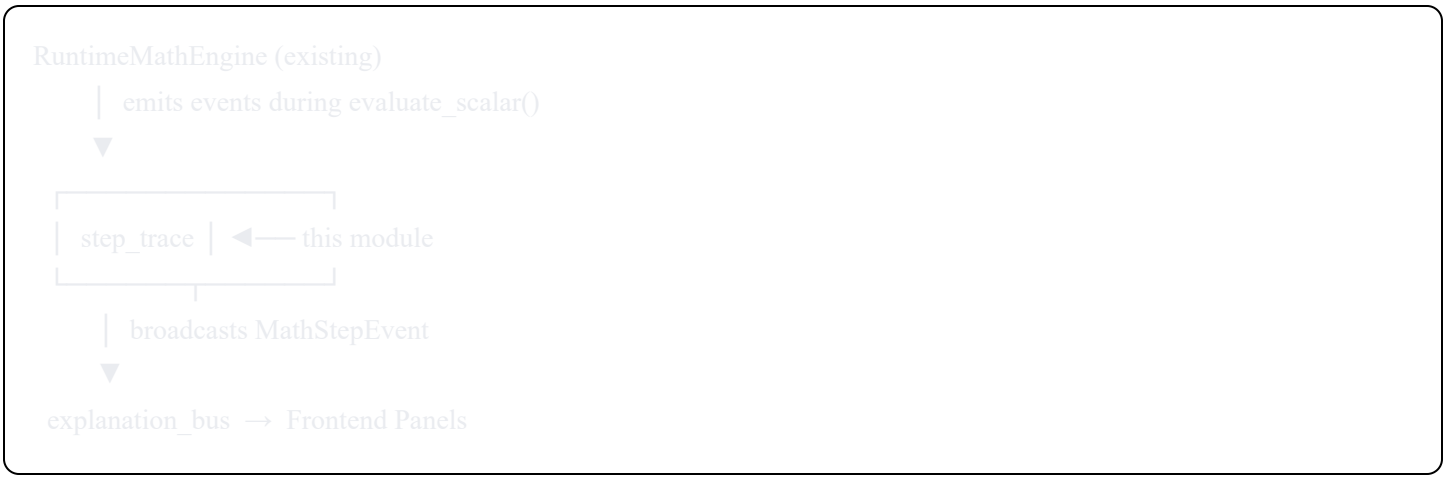
Module: `step_trace/`

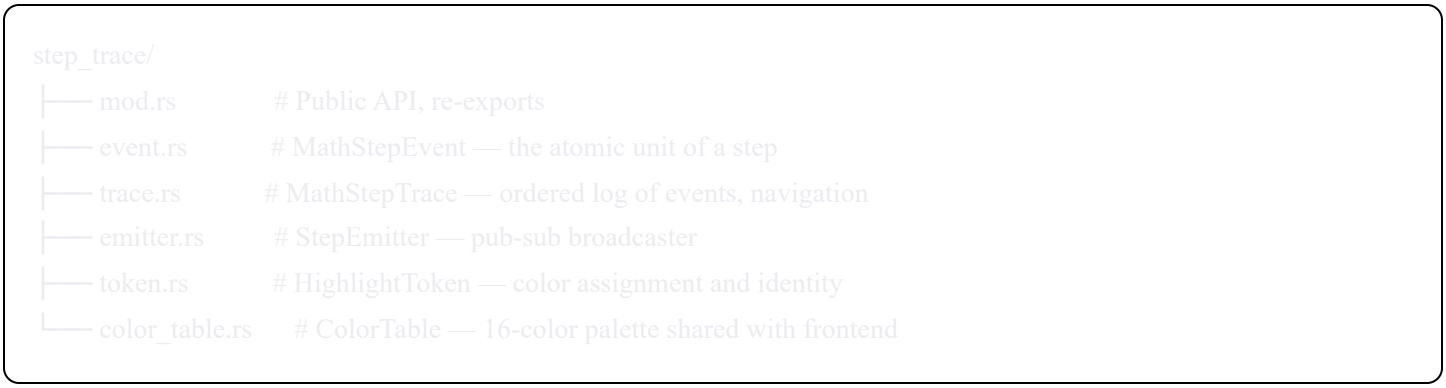
Step Trace & Equation Highlight System

Position in Architecture



This module is the instrumentation layer. It sits between the math engine and everything above it. It has no upstream dependencies on any new module — it only depends on existing types from `dsl/ast.rs` (for `node_id` values) and produces events that the `explanation_bus` and `compiler/runtime/snapshot_builder.rs` consume.

Directory Layout



File: `step_trace/event.rs`

Purpose

Defines `MathStepEvent` — one emitted record per sub-expression evaluation.

What it contains

`MathStepEvent` struct:

- `node_id: String` — the `AnnotatedExpr.node_id` from `dsl/ast.rs` for the expression node being evaluated. This is the stable identifier shared between the AST, the IR, the runtime, and the frontend.
- `highlight_token: Option<String>` — the pedagogically assigned token if this node is a designated teaching focus. Comes from `AnnotatedExpr.highlight_token`. Most nodes have `None`; only nodes the LLM marked as significant carry a token.
- `operation: StepOperation` — enum describing what mathematical operation produced this result.
- `left_value: Option<f64>` — left operand if binary. `None` for unary and function calls.
- `right_value: Option<f64>` — right operand if binary. `None` otherwise.
- `result_value: f64` — the computed numerical result of this node.
- `variable_bindings: Vec<(String, f64)>` — snapshot of all variable values active at the moment of this evaluation. Used by the Equation Panel to show substitution values.
- `description: String` — plain-English sentence describing this step. Examples: "Substituting θ = 1.57 into sin(θ)", "Multiplying the outer derivative 2u by the inner derivative cos(x)", "Adding 3.14 and 0 to get 3.14".
- `rule_applied: Option<String>` — name of the mathematical rule if one was applied. Examples: "Chain Rule", "Product Rule", "L'Hôpital's Rule". `None` for arithmetic operations.
- `is_finite: bool` — false if the result is NaN or infinite, triggering a warning display in the panel.
- `step_index: usize` — the global index of this event within the current `MathStepTrace`. Set by the `StepEmitter` when the event is registered.

`StepOperation` enum:

- `Add`, `Subtract`, `Multiply`, `Divide`, `Power` — binary arithmetic
- `Negate` — unary negation
- `FunctionCall { name: String }` — sin, cos, exp, ln, sqrt, etc.
- `Substitute { variable: String }` — replacing a variable with its current value
- `Derivative { variable: String, order: usize }` — derivative application
- `Integral { variable: String }` — integral evaluation
- `Limit { variable: String, approach: f64 }` — limit evaluation
- `Constant { name: String }` — evaluating π, e, etc.

Interfaces

- `MathStepEvent::new(node_id, operation, result, bindings) -> Self` — primary constructor, description auto-generated from operation.
- `MathStepEvent::with_highlight(mut self, token: String) -> Self` — builder method to attach a highlight token.
- `MathStepEvent::with_rule(mut self, rule: String) -> Self` — builder method to attach a rule name.
- `MathStepEvent::generate_description(&self) -> String` — private, called by constructor. Formats a human-readable sentence from the operation type and values.

Dependencies

- `dsl/ast.rs` — for `StepOperation` naming (no direct import, just mirrored names)
- Standard library only

File: `step_trace/trace.rs`

Purpose

`MathStepTrace` is the ordered, navigable log of all `MathStepEvent` objects emitted during one complete evaluation of a `MathExpression`. It is the data structure the Equation Execution Panel in the frontend reads to render the step history.

What it contains

`MathStepTrace` struct:

- `events: Vec<MathStepEvent>` — all events in emission order. Index 0 is the first sub-expression evaluated (deepest leaf), and the last index is the root expression's result.
- `current_index: usize` — the index of the currently active/displayed step. Starts at 0 and advances as the user presses "Next Step."
- `equation_id: Option<String>` — the `NamedEquation` ID from the `concept_library` module, if this trace corresponds to a named equation. `None` for ad-hoc evaluations.
- `total_steps: usize` — `events.len()`, cached for quick access.
- `is_complete: bool` — true once the root expression has been evaluated and all events are in the log.

Navigation methods:

- `advance(&mut self) -> Option<&MathStepEvent>` — moves `current_index` forward by 1 and returns the now-active event. Returns `None` if already at end.

- `rewind(&mut self) -> Option<&MathStepEvent>` — moves `current_index` backward by 1. Returns `None` if already at start.
- `jump_to(&mut self, index: usize) -> Option<&MathStepEvent>` — seeks to a specific index. Used by the scrubber control.
- `current(&self) -> Option<&MathStepEvent>` — returns the event at `current_index`.
- `active_highlight_token(&self) -> Option<&str>` — returns the `highlight_token` of the current event, or `None`. This is the primary value read by the `SnapshotBuilder` to populate `RendererSnapshot.active_highlight_token`.
- `history_up_to_current(&self) -> &[MathStepEvent]` — returns the slice `events[0..=current_index]`, used by the Equation Panel to render the step history list.
- `reset(&mut self)` — sets `current_index = 0`.

Recording methods (called by StepEmitter):

- `push_event(&mut self, event: MathStepEvent)` — appends an event, sets its `step_index`, increments `total_steps`.
- `mark_complete(&mut self)` — sets `is_complete = true`.

Serialization

`MathStepTrace` derives `Serialize` and `Deserialize` so the full trace can be sent over WebSocket to the frontend for client-side scrubbing without additional server round-trips.

Dependencies

- `step_trace/event.rs` — `MathStepEvent`

File: `step_trace/token.rs`

Purpose

`HighlightToken` pairs a short string token identifier with a color index. It is the shared identity that ties together: a span in the rendered KaTeX equation, a span in the narrative prose, and a 3D object in the scene.

What it contains

`HighlightToken` struct:

- `token: String` — short alphanumeric identifier, e.g. `"hk_03"`. Assigned by the `LLMOrchestrator`. Format: `"hk_"` prefix + two-digit zero-padded index.
- `color_index: u8` — index 0–15 into the `ColorTable`. The same index is used by both the frontend CSS (for equation spans and prose spans) and by the Three.js backend (for emissive material color).
- `display_name: String` — human-readable name of what this token represents. Examples: "outer function", "inner function", "exponent", "coefficient". Shown in the "Why this step?" drawer.

`HighlightTokenRegistry` struct:

- `tokens: HashMap<String, HighlightToken>` — maps token string to `HighlightToken`.
- `register(token: String, color_index: u8, display_name: String) -> &HighlightToken` — adds a token to the registry.
- `get(token: &str) -> Option<&HighlightToken>` — lookup by token string.
- `color_for(token: &str) -> Option<u8>` — convenience lookup for color index only.
- `clear()` — resets the registry when a new concept session begins.

The registry is populated by the `LLMOrchestrator` when it processes a new `OrchestratorResponse`. It reads the `AnimationIntent`'s highlight token assignments and registers each one. The registry instance is stored in the session context and passed by reference into the snapshot builder.

Dependencies

- Standard library only

File: `step_trace/color_table.rs`

Purpose

Defines the 16-color palette shared between the Rust backend and the JavaScript frontend. Both sides must use identical color values so an orange highlight on an equation term produces the same orange glow on the Three.js mesh.

What it contains

`ColorTable` struct:

- `entries: [ColorEntry; 16]` — fixed-size array of 16 entries.

`ColorEntry` struct:

- `index: u8` — 0–15
- `name: &'static str` — human-readable name: "amber", "teal", "rose", "violet", "sky", "lime", "orange", "cyan", "pink", "indigo", "emerald", "red", "yellow", "blue", "purple", "green"
- `hex: &'static str` — CSS hex string, e.g. `"#F59E0B"` for amber
- `rgb: [u8; 3]` — RGB triple for use in Three.js material emissive color
- `css_class: &'static str` — CSS class name applied to `<span>` elements in the frontend, e.g. `"highlight-amber"`

`ColorTable::get(index: u8) -> &'static ColorEntry` — panics if index > 15 (programmer error, not user error).

`ColorTable::for_token(token: &str, registry: &HighlightTokenRegistry) -> Option<&'static ColorEntry>` — convenience method.

`ColorTable::as_json() -> serde_json::Value` — serializes the full table as JSON. This JSON is embedded into the frontend JavaScript bundle at build time so the browser has the same color values as the Rust backend.

Why a fixed table

The color table must be identical on both sides of the Rust/WASM boundary. Hard-coding it as a `const` array in Rust and exporting it as JSON for the frontend build step is the only approach that guarantees consistency without a runtime synchronization step.

File: `step_trace/emitter.rs`

Purpose

`StepEmitter` is the pub-sub broadcaster that the `RuntimeMathEngine` calls during expression evaluation. It collects events into the current `MathStepTrace` and notifies registered subscribers.

What it contains

`StepEmitter` struct:

- `current_trace: MathStepTrace` — the trace currently being built. Reset at the start of each new expression evaluation.
- `subscribers: Vec<Box<dyn StepSubscriber>>` — registered listeners. The `ExplanationBus` is the primary subscriber; additional subscribers could include logging or analytics.
- `token_registry: Arc<RwLock<HighlightTokenRegistry>>` — shared reference to the session's token registry, used to look up the color index for events with `highlight_token` set.

`StepSubscriber` trait:

```
trait StepSubscriber: Send + Sync {
    fn on_step(&mut self, event: &MathStepEvent, trace: &MathStepTrace);
    fn on_trace_complete(&mut self, trace: &MathStepTrace);
}
```

Public methods:

- `StepEmitter::new(token_registry: Arc<RwLock<HighlightTokenRegistry>>) -> Self`
- `subscribe(&mut self, subscriber: Box<dyn StepSubscriber>)` — adds a listener.
- `begin_trace(&mut self, equation_id: Option<String>)` — called by `RuntimeMathEngine` before evaluating an expression. Resets `current_trace` with a fresh `MathStepTrace`.
- `emit(&mut self, event: MathStepEvent)` — called by `RuntimeMathEngine` after each node evaluation. Pushes the event to `current_trace`, then calls `on_step()` on all subscribers.
- `end_trace(&mut self)` — called by `RuntimeMathEngine` after the root expression returns. Marks the trace complete and calls `on_trace_complete()` on all subscribers.
- `current_trace(&self) -> &MathStepTrace` — read-only access to the in-progress or last-completed trace. Used by `RuntimeEngine` to read `active_highlight_token` for state update.
- `take_trace(&mut self) -> MathStepTrace` — takes ownership of the completed trace and resets the emitter for the next evaluation.

Thread safety: The emitter is wrapped in `Arc<Mutex<StepEmitter>>` by the `RuntimeEngine` and `RuntimeMathEngine`. This allows the `RuntimeEngine` to both inject it into the math engine and also read the current trace's active token after each evaluation without a separate channel.

Calling sequence (from `RuntimeMathEngine.evaluate_expression`)

```
emitter.begin_trace(equation_id)
    └── evaluate_scalar(root)
        ├── evaluate_scalar(left_child)
        │   ├── evaluate_scalar(leaf)
        │   │   └── emitter.emit(leaf_event)
        │   └── emitter.emit(left_event)
        └── evaluate_scalar(right_child)
            └── emitter.emit(right_event)
        └── emitter.emit(root_event)
emitter.end_trace()
```

This produces events in post-order (leaves first, root last), which is the natural mathematical evaluation order. The Equation Panel displays them in this order: the innermost computation is shown first, building up to the

final result.

File: `step_trace/mod.rs`

Purpose

Public API and re-exports.

What it contains

- Re-exports: `MathStepEvent`, `StepOperation`, `MathStepTrace`, `HighlightToken`, `HighlightTokenRegistry`, `ColorTable`, `ColorEntry`, `StepEmitter`, `StepSubscriber`
- Module-level documentation explaining the data flow
- Feature flag: `#[cfg(feature = "step-trace")]` guard (optional — allows disabling for performance benchmarks)

Dependencies Summary

Depends on	Why
`dsl/ast.rs` (node_id types)	Node IDs originate in the AST and flow through to events
`compiler/runtime/src/math/runtime_engine.rs`	Calls `emit()` during evaluation — this is where events originate
Standard library (`std::sync`, `std::collections`)	Thread safety, storage
`serde`	Event and trace serialization for WebSocket transmission

Consumed by

Consumer	What it reads
`explanation_bus/`	Subscribes to `StepEmitter`, receives `MathStepEvent` on each step
`compiler/runtime/engine.rs`	Reads `current_trace().active_highlight_token()` after each evaluation to update `RuntimeState.current_highlight_token`
`compiler/runtime/snapshot_builder.rs`	Reads the completed `MathStepTrace` to embed in `RendererSnapshot`

Consumer	What it reads
Frontend Equation Execution Panel	Receives serialized `MathStepTrace` over WebSocket